GALILEO Group 31 october 2025 Deadline Report

Francesco Pivotto 2158296
Giorgia Amato 2159999
Alessio Demo 2142885

10 novembre 2025

Indice

1 Introduction

2 Baseline NL

The Baseline NL module provides a reference framework for evaluating Large Language Models (LLMs) through direct interaction via natural language prompts. This approach assesses the model's intrinsic reasoning and knowledge by prompting it with natural language (NL) questions, optionally enriched with structural and contextual information derived from a database schema or a specific query results. All outputs are standardized in a unified FULL JSON format to facilitate downstream evaluation.

2.1 General Procedure

1. **Prompt Loading:** Natural language prompts are loaded from dataset-specific resources (**queries_<dataset-name>.txt**). Each prompt corresponds to a query from the same dataset.

2. **LLM interaction:** Prompts are processed through a **LangChain-based pipeline**, which integrates:

   - The general task definition and LLM role specification.
   - The corresponding database schema, extracted via DuckDB.
   - Optional contextual data (e.g. query results for Model Context datasets).
   - The natural language question itself

3. **Response Parsing and Storage:** The model's output is parsed using a **PydanticOutputParser**, ensuring compliance with the required FULL JSON format. Each response is stored in an individual JSON file, forming a consistent source for evaluation.

2.2 Dataset Categorization and Contextualization

1. **Internal knowledge:** In these datasets, the model relies exclusively on its pretrained knowledge and linguistic reasoning capabilities. The prompt includes only the natural language question and, optionally, the database schema, but no factual data retrieved from the database.

2. **Model Context:** These datasets implement a **Retrieval-Augmented Generation (RAG)-inspired** mechanism. Prior to querying the LLM, the corresponding SQL query is executed against the database, and the retrieved tuples are supplied to the model as external contextual input. This design allows the model to reason jointly over its internal representations and the retrieved data, thus improving factual consistency and grounding. Each resulting file produced by the module fits the following FULL JSON format:

```
{
    "result_set": [
        { "originaltitle": "The Three Musketeers" },
        { "originaltitle": "The Count of Monte Cristo" }
```

```
    ],
    "time": 1.84,
    "tokens": 312
}
```

where:

- **result_set:** Denotes the structured LLM output.
- **time:** Represents the total time taken by the LLM to generate the response.
- **tokens:** Indicates the number of tokens processed during the interaction

2.3 Architecture:

The architecture is composed of three main components:

1. **Configuration Layer:**
   - API keys for the supported providers (**IBMWatsonx** and **Google Gemini**) are loaded form a **.env** file.
   - The **Config_Loader** selects the target LLM provider, and the appropriate wrapper is instantiated through **get_llm_wrapper()**.
   - Each wrapper extends the **LLMBaseWrapper** class, implementing provider-specific logic for model initialization, token counting, and response handling.
2. **Chain Construction:**

   The **build_lcel_chain()** method constructs a **LangChain Expression Language (LCEL)** pipeline comprising:
   - **Database Context Retrieval: get_db_context()** extracts schema information and, for Model Context datasets, executes the original SQL query using DuckDB database instance of corresponding dataset.
   - **Prompt Composition: ChatPromptTemplate** merges two layers, a **SYSTEM_PROMPT** defining the LLM's role and behavior, and a **HUMAN_PROMPT** containing the NL question and formatting directives.
   - **Structured Output Parsing:** The **PydanticOutputParser** enforces adherence to the predefined FULL JSON schema.
3. **Execution Flow**
   - For each dataset, SQL queries and corresponding NL prompts are loaded.
   - The **(prompt, query)** pairs are processed through **chain.invoke()**.
   - Responses are parsed, validated, and augmented with the metadata such as execution time and token usage.
   - The resulting files are serialized via **save_baseline_to_json()** for evaluation purposes.

2.4 Results and Evaluation

The results obtained from the Baseline NL module were evaluated using standard metrics to assess the performance of the LLMs in both Natural Language (NL) and SQL contexts. The model used for the **gemini** provider is: **gemini-2.5-flash**, instead for **IBMWatsonx** provider we used **meta-llama/llama-3-3-70b-instruct**. All systems are tested on the datasets described in Table 2 of the previous project task, excluding Premier and Fortune like the galois authors did.

Metric	**NL**	**SQL**
F1-CELL	0.391	0.537
CARDINAITY	0.644	0.733
TUPLE-CONSTR.	0.182	0.331
AVG-SCORE	0.406	0.534
#TOKENS(M)	39965	126946
AVG-TIME	4.812	7.561

Tabella 1: GEMINI results

Metric	**NL**	**SQL**
F1-CELL	0.443	0.338
CARDINAITY	0.693	0.577
TUPLE-CONSTR.	0.209	0.121
AVG-SCORE	0.448	0.345
#TOKENS(M)	14808	9090
AVG-TIME	4.691	1.192

Tabella 2: WATSONX results